

```
docker build -t mygpt2-textgen .
```

```
# Run the container
docker run -p 5000:5000 mygpt2-textgen
```

To test the implementation in a new terminal, send a request using *curl*:

```
curl -X POST http://localhost:5000/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Once upon a time"}'
```

The JSON response text will be generated as the output. Here's another example of Dockerizing a GPT-based text generator.

The Dockerfile is:

```
FROM python:3.10

# Install dependencies
RUN pip install --no-cache-dir transformers torch flask

# Copy application code
workingdir /app
COPY app.py .

# Run the web server
CMD ["python", "app.py"]
```

To create *app.py*, use the following code:

```
from flask import Flask, request, jsonify
from transformers import pipeline

app = Flask(__name__)
generator = pipeline("text-generation", model="gpt2")

@app.route("/generate", methods=["POST"])
def generate():

    myprompt = request.json.get("prompt", "")
    result = generator(myprompt, max_length=50, num_return_
sequences=1)

    return jsonify(myresult)
```

Now build and run the container:

```
docker build -t gpt-text-gen .
docker run -p 5000:5000 gpt-text-gen
```

You can now send POST requests to *localhost:5000/generate*.

GPU support for really large language models

To run models like Llama or Stable Diffusion efficiently, use GPU-enabled containers.

The requirements are:

- Docker + NVIDIA container toolkit
- NVIDIA GPU
- Base image like NVIDIA CUDA

The Dockerfile is:

```
FROM nvidia/cuda:11.8.0-cudnn8-runtime-ubuntu22.04
```

```
RUN apt update && apt install -y python3-pip
```

```
RUN pip install torch torchvision transformers diffusers
```

Next, run:

```
docker run --gpus all my-image-name
```

Best practices when integrating Docker with genAI include:

- Make use of builds with multiple stages to keep container images light.
- Utilise layer-based model caching to avoid recurring downloads.
- Integrate environment variables for model parameters and configurations.
- For production, use Gunicorn, Uvicorn, or Triton Inference Server.
- Use Prometheus and Grafana to track resource consumption.

GenAI applications can be managed, shared and scaled more easily thanks to the containerisation of AI components by Docker, which addresses the issues with building and running AI models locally and in production.

Using Docker to containerise generative AI applications is a good way to make development, deployment, and scaling of applications easier. Docker lets you focus on innovation without worrying about system-level inconsistencies, whether you're making an LLM-powered chatbot or a GPU-intensive image generator. As AI continues to evolve, so will the infrastructure surrounding it. Docker is proving to be a cornerstone of that ecosystem. **END** 🐧

 **By: Dr Gaurav Kumar**

The author is associated with various academic and research institutes for delivering expert lectures and conducting technical workshops on the latest technologies and tools.